

Wie niet sterk is, moet slim zijn

Pauli propagation at 10^7 terms: how a GF(2)-linear hash turned a memory-bound simulator strong and smart.

Contents

01 Sterk — brute force against an exponential wall

02 The wall is memory

03 Sterk én slim — a hash that respects XOR

04 The gift that keeps on giving

01

Sterk

Pay for the operator, not the state

- State vectors carry 2^n amplitudes — exact to about thirty qubits, then nothing.
- Tensor networks pay for entanglement, not for qubits.
- Pauli propagation evolves the observable and truncates the sum.

The whole method on one line

$$U^\dagger O U = \sum_P c_P P, \quad P \in \{I, X, Y, Z\}^{\otimes n}$$

Evolve O backwards through the gates, drop the coefficients that do not matter, contract with ρ_0 at the end. Exactness traded for sparsity.

**It reproduces the 127-qubit IBM utility observables,
and it is bit-exact against stim at the Clifford points.**

— Kim et al. 2023; `benchmarks/python/bench_a_clifford.py`

A Pauli string is two bit vectors

	I	X	Z	Y
x	0	1	0	1
z	0	0	1	1

Two words per string, x : $[u64; w]$ and z : $[u64; w]$. A product is an XOR of both and a phase i^k ; two strings commute exactly when $x_1 \cdot z_2 + z_1 \cdot x_2 = 0$.

No pointers, no allocation, Copy.

48 B

One term at 128 qubits: two sixteen-byte keys and a complex coefficient. Remember that number — every argument in this talk is a multiple of it.

A Clifford gate is a permutation

GATE	IN	OUT
H	X, Z	Z, X
S	X, Y	$Y, -X$
CNOT	X_c	$X_c X_t$
CNOT	Z_t	$Z_c Z_t$

One string in, one string out. In bits the key changes only on the gate’s support, by an XOR with a vector from a small delta set D .

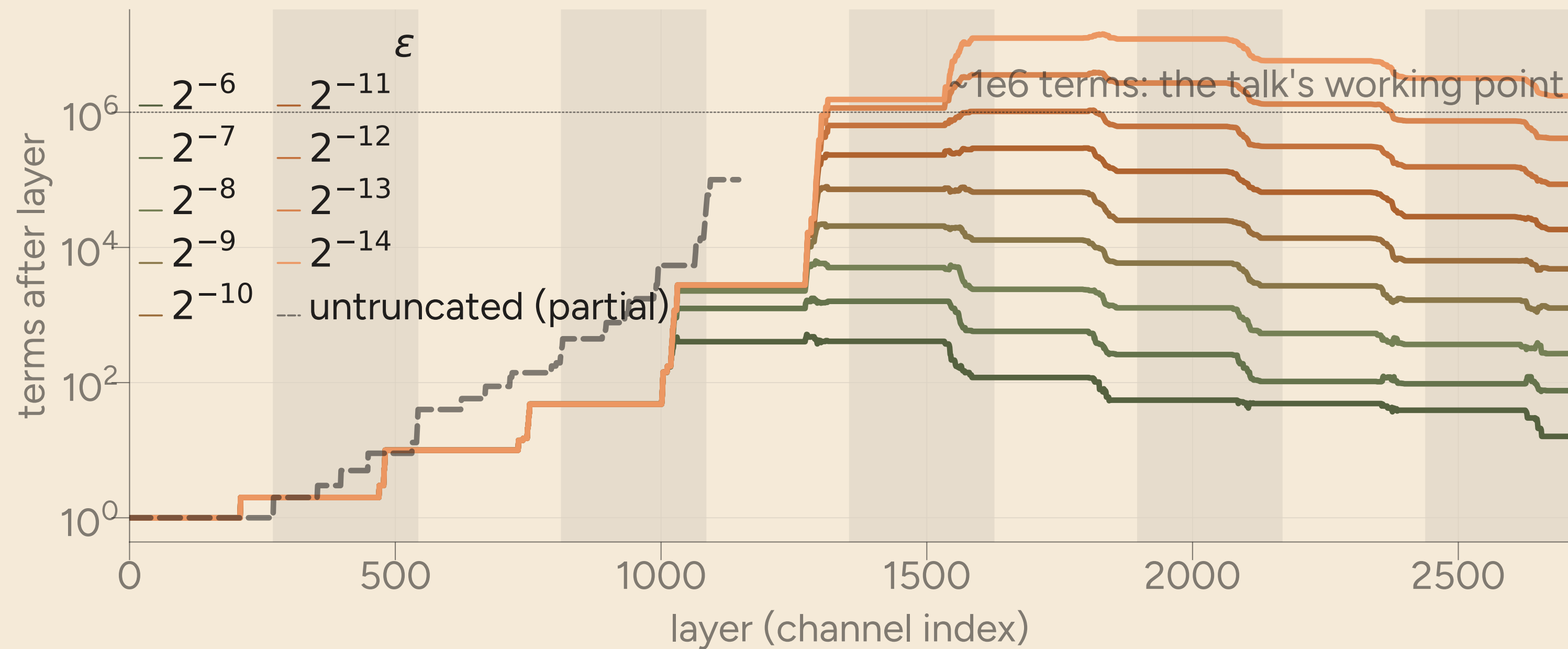
Term count is invariant. Truncation never fires. Clifford circuits are free.

Beyond Clifford, one string becomes two

$$R^\dagger Q R = \begin{cases} Q & [P, Q] = 0 \\ \cos \theta Q + \sin \theta (iPQ) & \{P, Q\} = 0 \end{cases}$$

The anticommuting part doubles every layer. Truncation is what keeps the sum finite: drop small coefficients, drop high weight, keep the top N .

Three billion term updates



127-qubit heavy-hex kicked Ising, observable Z_{62} , Heisenberg. Terms surviving after each channel, one curve per truncation threshold.

The working point

2710

rotations — ten Trotter steps on
127 qubits

1.07 M

terms at the peak, at $\varepsilon = 2^{-12}$

$3 \cdot 10^9$

term updates in one run

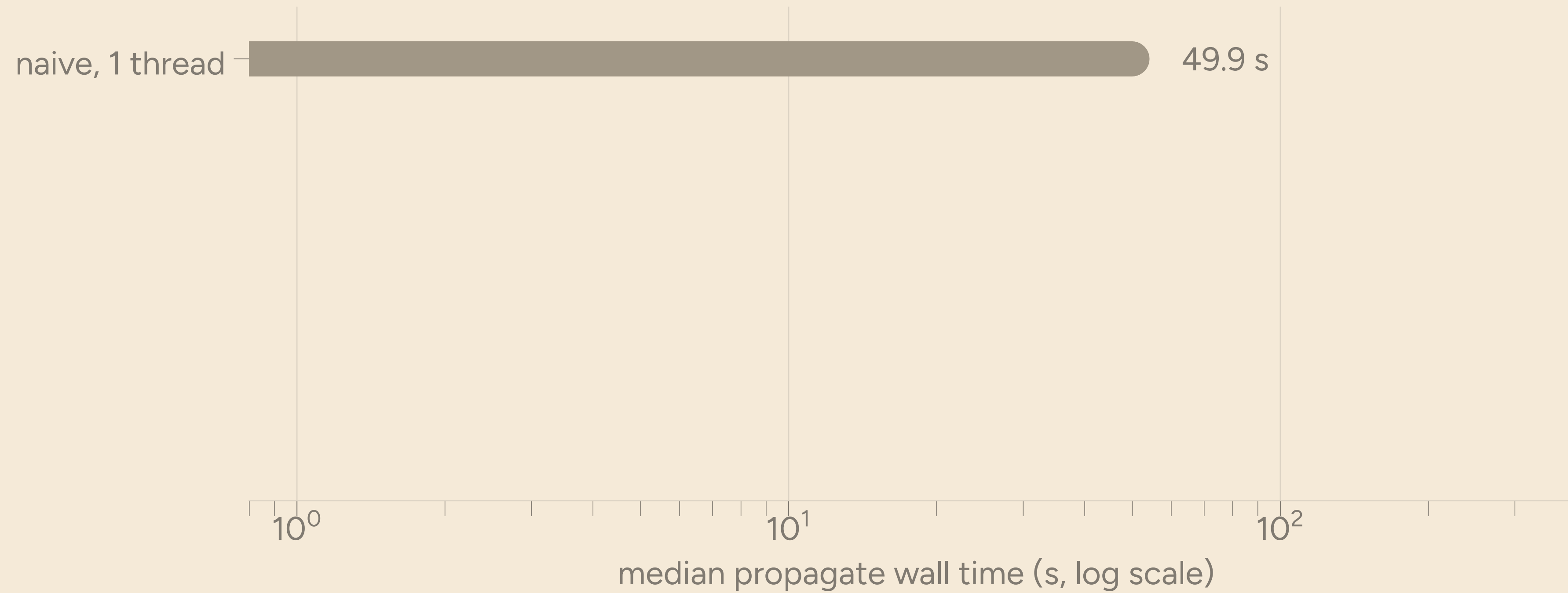
A CPU does 10^{11} simple operations per second. This should take three seconds.

The naive algorithm

```
for gate in circuit.reversed():
    out = {}
    for (P, c) in sum:
        for (P', c') in gate.apply(P, c):
            out[P'] += c'
    sum = {P: c for P, c in out if |c| >= eps}
```

Or with arrays: emit every output, sort, merge equal keys, filter. Either way, one global order per layer.

Forty-nine seconds

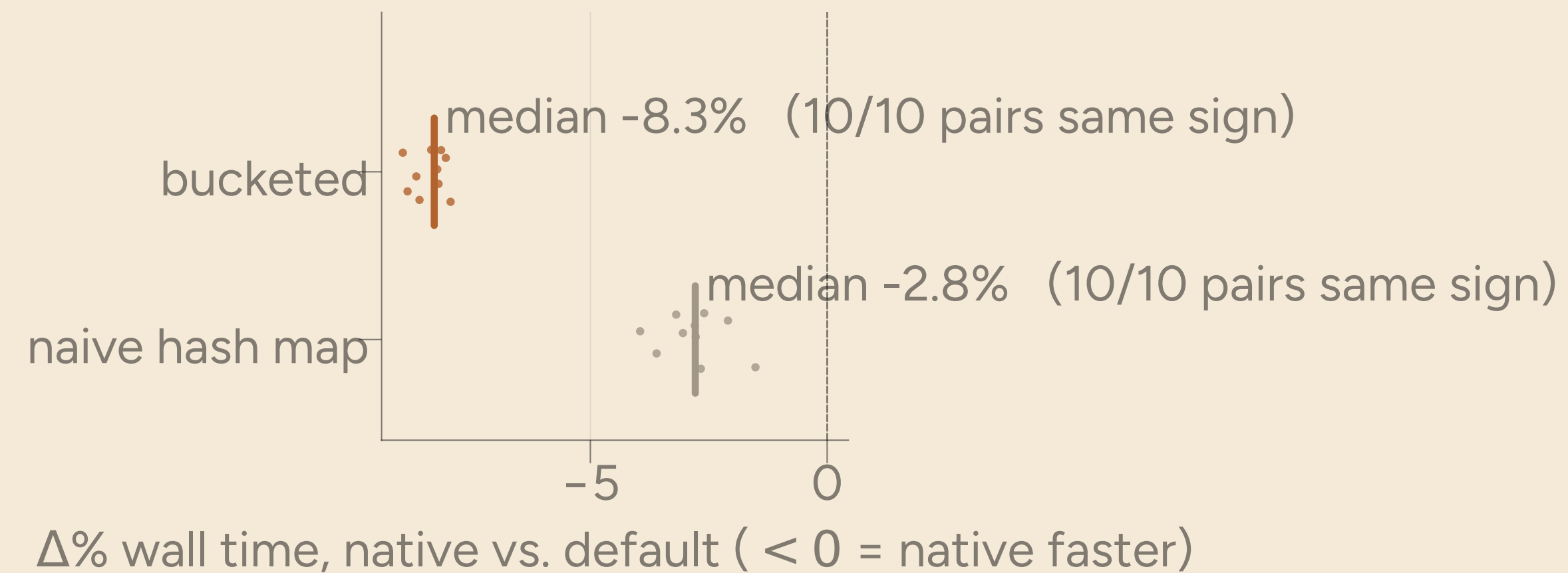


Median wall time for the 2710-channel circuit. This chart returns after every idea.

Being strong: make the kernel fast

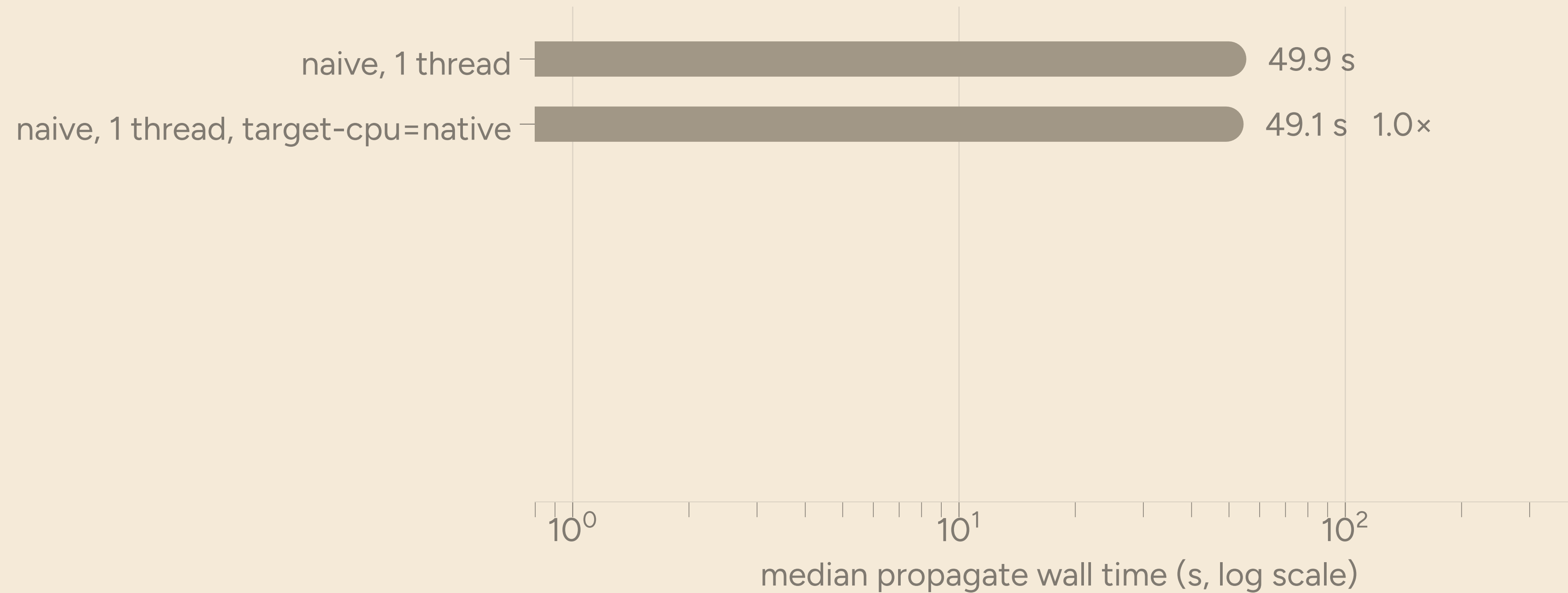
- A fast non-cryptographic hash — FxHash, not SipHash.
- Popcount and XOR are already single instructions.
- Let LLVM vectorize: `-C target-cpu=native`, fat LTO, one codegen unit.

A real gain, a hundred times too small



Paired runs, alternated abba, one thread: 3 % on the hash map, 8 % on the sorted engine — consistent in every pair, and nowhere near enough.

The ladder, after being strong



Tuning the kernel moves the bar by a few percent. The arithmetic was never the cost.

02

The wall is memory

Count the bytes, not the flops

- Read the sum, write the outputs, sort or probe them, write the merged result.
- At a million terms and fanout two that is 300–400 MB per layer.
- The arithmetic per term is a few XORs and one multiply — under a nanosecond.

Two ways to lose

A hash map is random access

Every probe into a fifty-megabyte table is a cache miss: about a hundred nanoseconds, one at a time, and the core waits for each one.

A sort is streaming

Every pass runs at memory bandwidth: about ten gigabytes per second per core, so thirty to forty milliseconds a layer just to move the bytes.

Measured bandwidth, reference host

	PLACEMENT	READ	WRITE	COPY
●	1 core	11.3	10.1	9.5
●	1 core, other socket's memory	7.8	7.2	5.5
●	8 cores, one socket	39.0	18.6	35.6
●	16 cores, both sockets	45.0	25.3	40.0
●	32 threads (SMT)	48.8	23.1	33.8

GB/s, STREAM-style, crates/membench. The spec sheet says 141 GB/s per socket: two of six memory channels are populated.

45 GB/s

Everything the machine has, shared by every core. At 48 bytes a term that is a hard ceiling of a billion term-moves per second, before any algorithm is written.

Hyperthreads add no bandwidth

- The second socket adds 15–25 %, not a factor of two — a remote read runs at 7.8 GB/s.
- Write bandwidth is half of read.
- Thirty-two threads move no more bytes than sixteen cores do.

Three levers, and a hard limit

Fewer bytes

Compact keys, no pointers, structure of arrays, coefficients that are not boxed.

More work per byte

Do everything a term needs while it is in cache. Locality, not clock speed.

More bandwidth

More cores, more sockets, more nodes — and the caches and capacity they bring.

The hard limit

10^7 terms is half a gigabyte; a global sort needs several times that. At 10^8 a node runs out of memory before it runs out of time.

What I tried first

Per-thread dictionaries

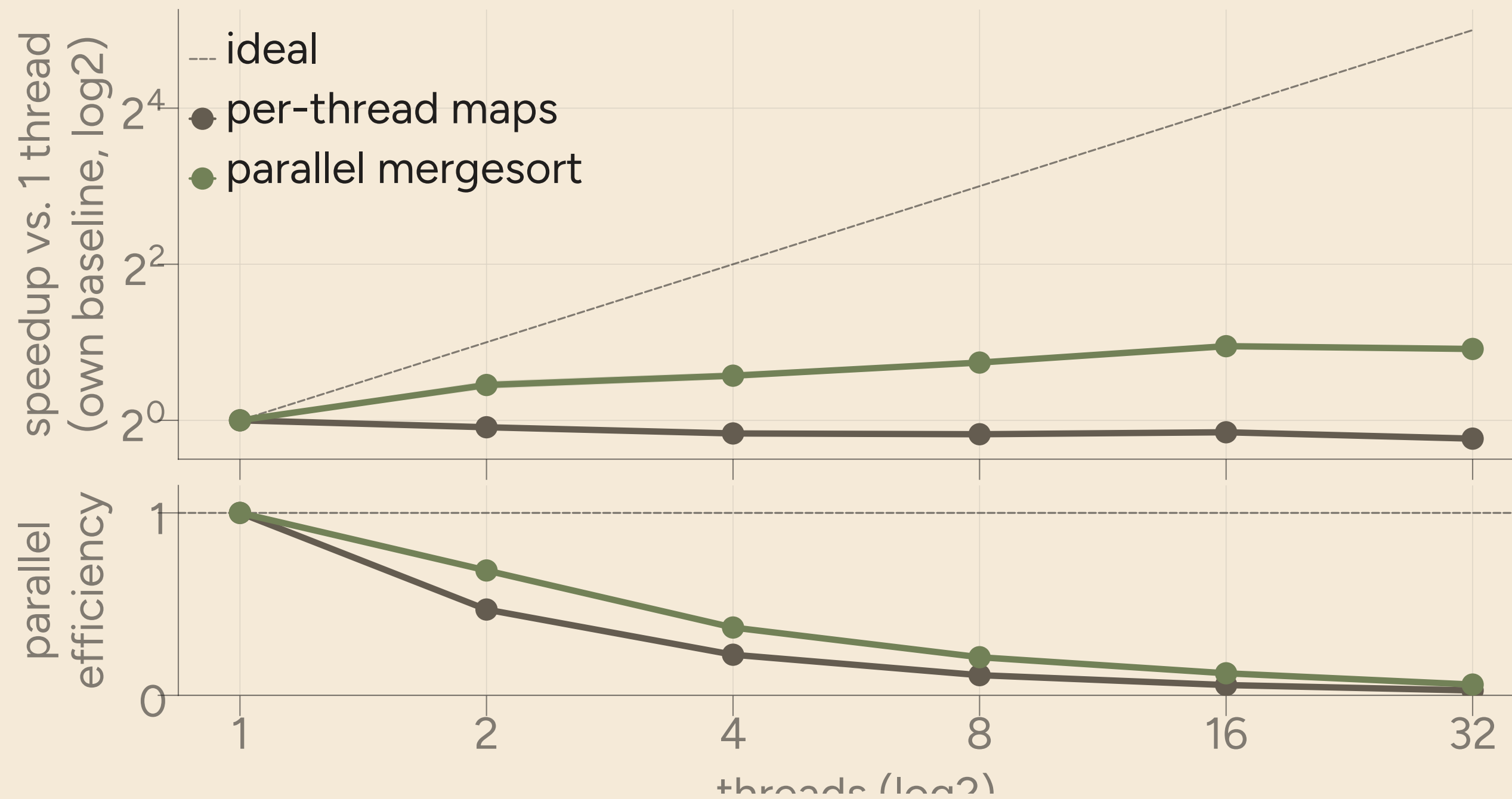
Split the terms into chunks, one hash map per thread, apply the gate locally — then merge every map into one at the end of the layer.

Parallel mergesort

One flat array of rows, every thread appends its outputs, then a parallel sort of the whole array and a merge pass for equal keys.

Both are what a competent engineer writes in an afternoon. Both keep one global order.

Thirty-two cores, a factor of two



Reconstructed in Rust on the same circuit, gated against the reference engine to 10^{-9} . Bands are min-max over repeated runs.

Amdahl does the rest

- Mergesort: 1.9× at sixteen threads, then flat.
- Per-thread maps: never faster than one thread — the final merge re-inserts every term.
- Memory per term goes up: 280 and 830 bytes against 240 for the plain map.

We were strong. We were not smart.

— every attempt kept one global order, and paid for it with one global merge

03

Sterk én slim

A hash that respects XOR

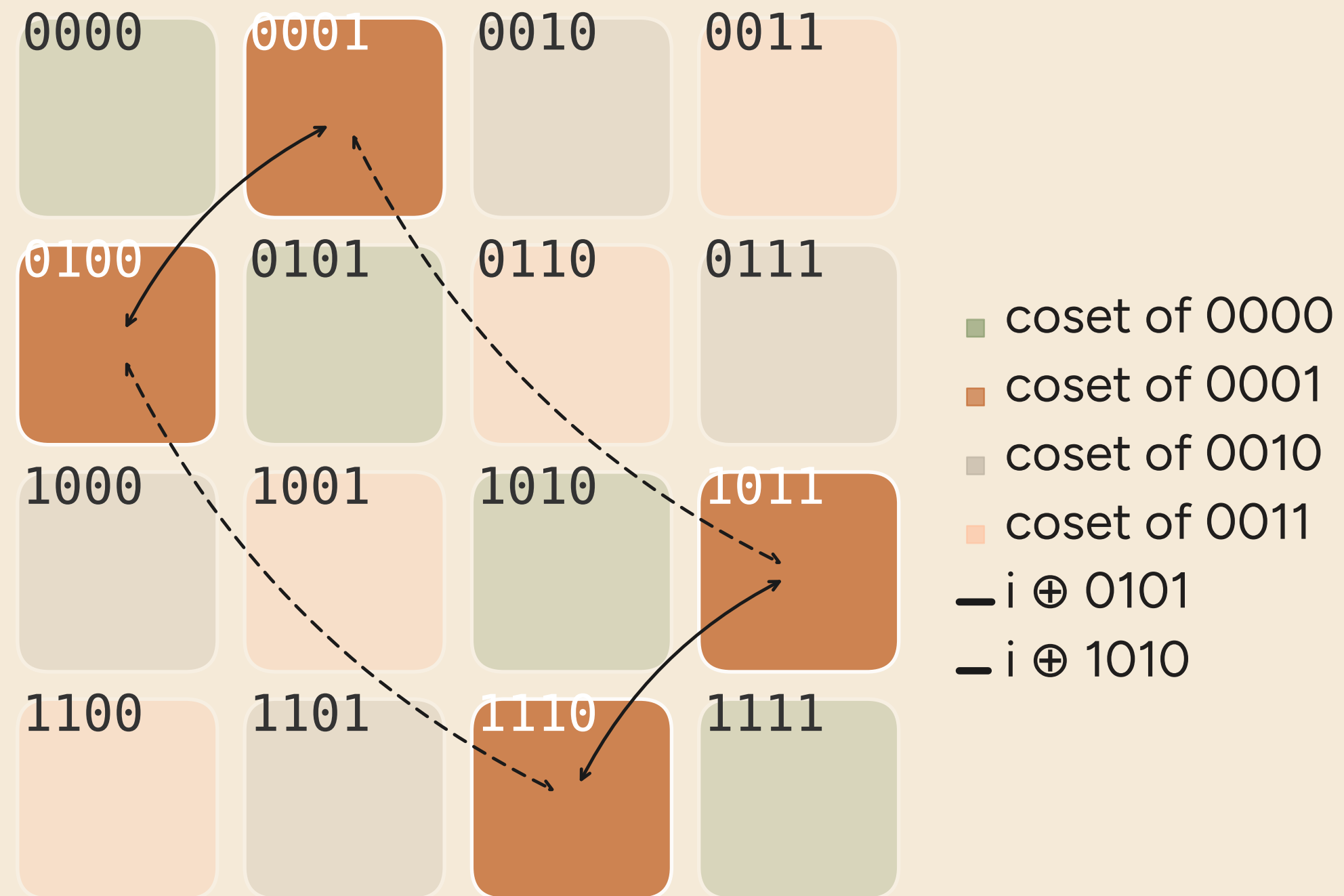
$$h(v) = Hv \pmod{2}, \quad h(v \oplus d) = h(v) \oplus h(d)$$

Partition the sum into $B = 2^b$ buckets by a random binary matrix H . A gate changes a key by an XOR with one of at most sixteen deltas, so a bucket's output bucket is known before any term is touched.

What linearity buys

- A bucket's outputs go to at most $2^{\dim D}$ statically known buckets.
- Two equal output keys have equal hashes: duplicates never straddle buckets.
- So deduplication is bucket-local, and there is no global sort anywhere.

Cosets are closed tasks



Sixteen buckets, $h(D)$ spanning two vectors: four closed cosets. Arrows trace one coset's entire data movement — every edge stays inside it.

No locks, no atomics, no barriers

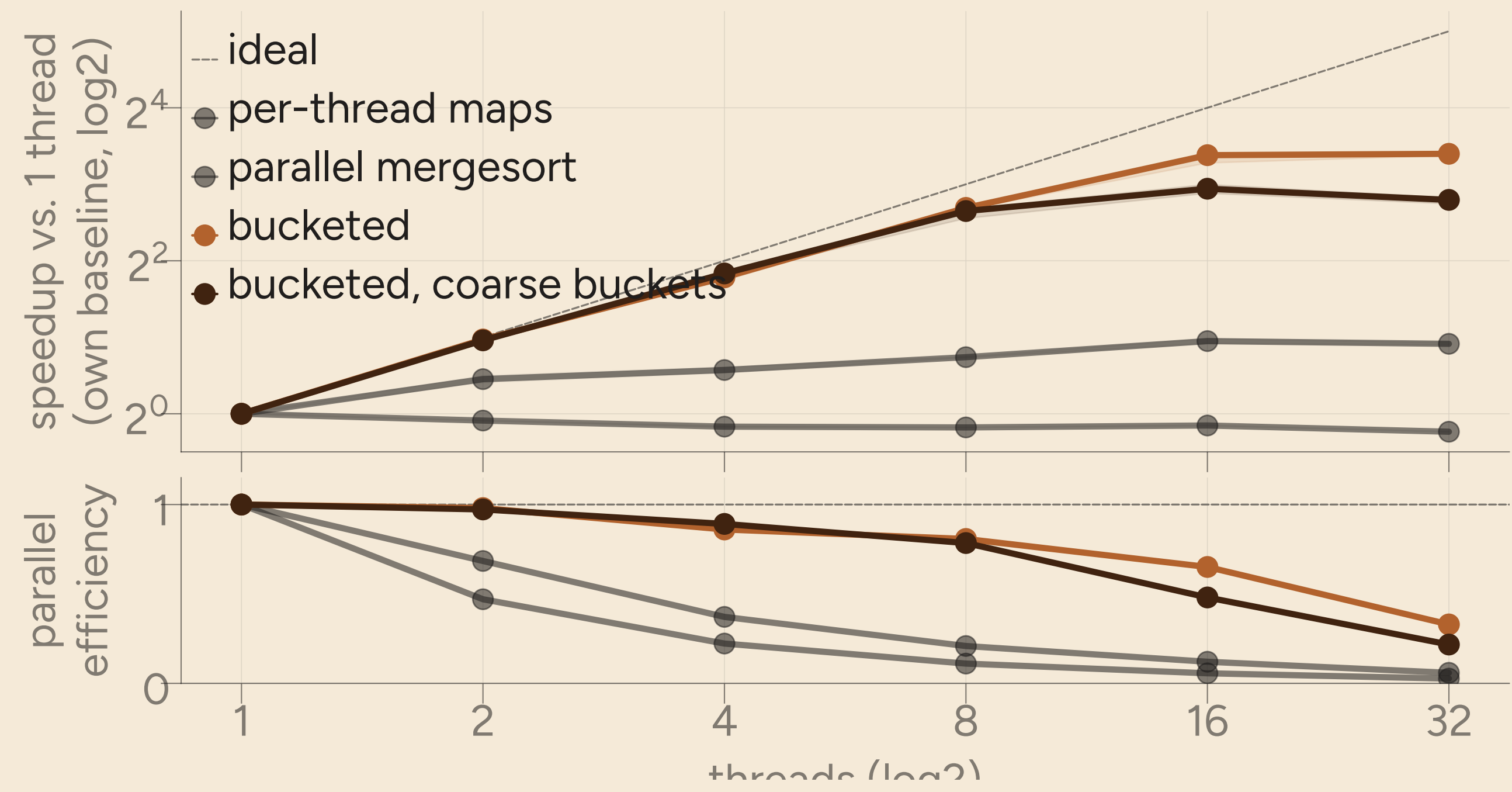
- A coset's outputs read only the same coset: an independent, write-disjoint task.
- Each task sorts and merges a cache-sized run.
- Rayon work-steals cosets. That is the whole parallel runtime.

Load balance comes from randomness

$$\max_{\beta} |\text{bucket}_{\beta}| \leq \frac{m}{B} + O\left(\sqrt{\frac{m \log B}{B}}\right)$$

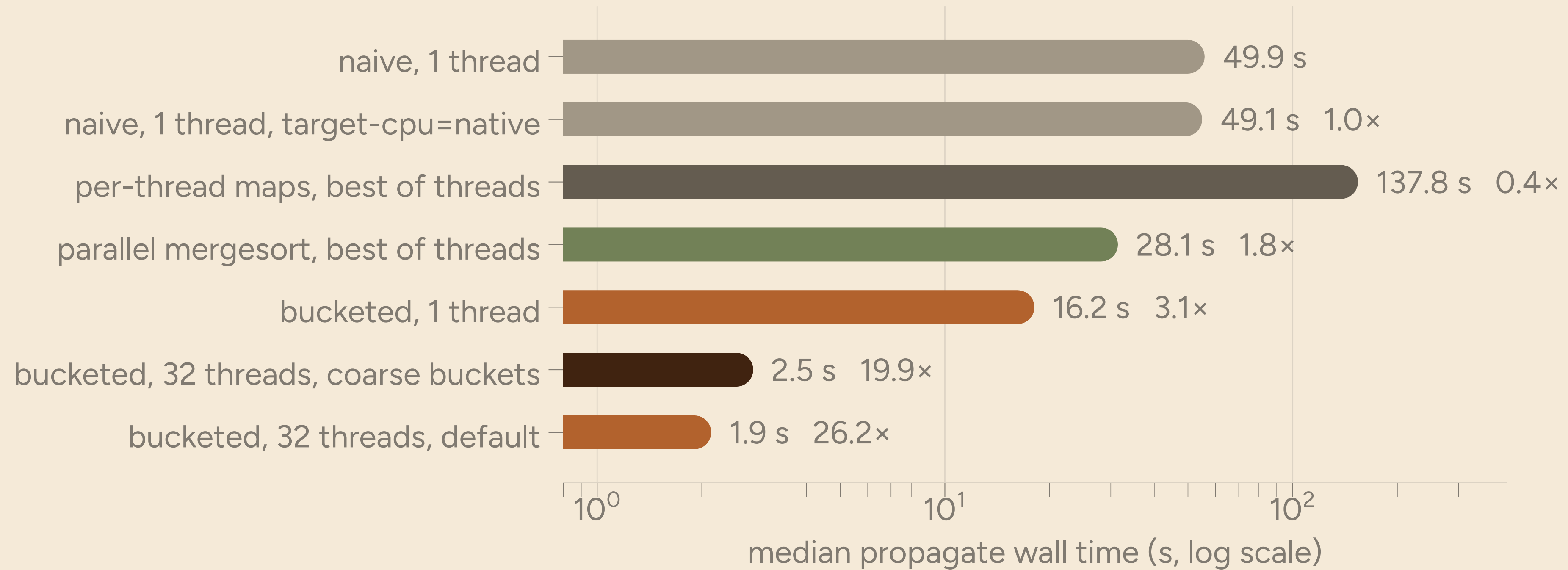
A dense random H is a universal hash family, so the bound holds whatever the input looks like. Truncated sums are extremely structured — a coordinate-based partition would put everything in one bucket. The random H does not care.

Where it ends up



Speedup against each variant's own single-thread run; the two reconstructions greyed. Sixteen physical cores, thirty-two hyperthreads.

The ladder, finished

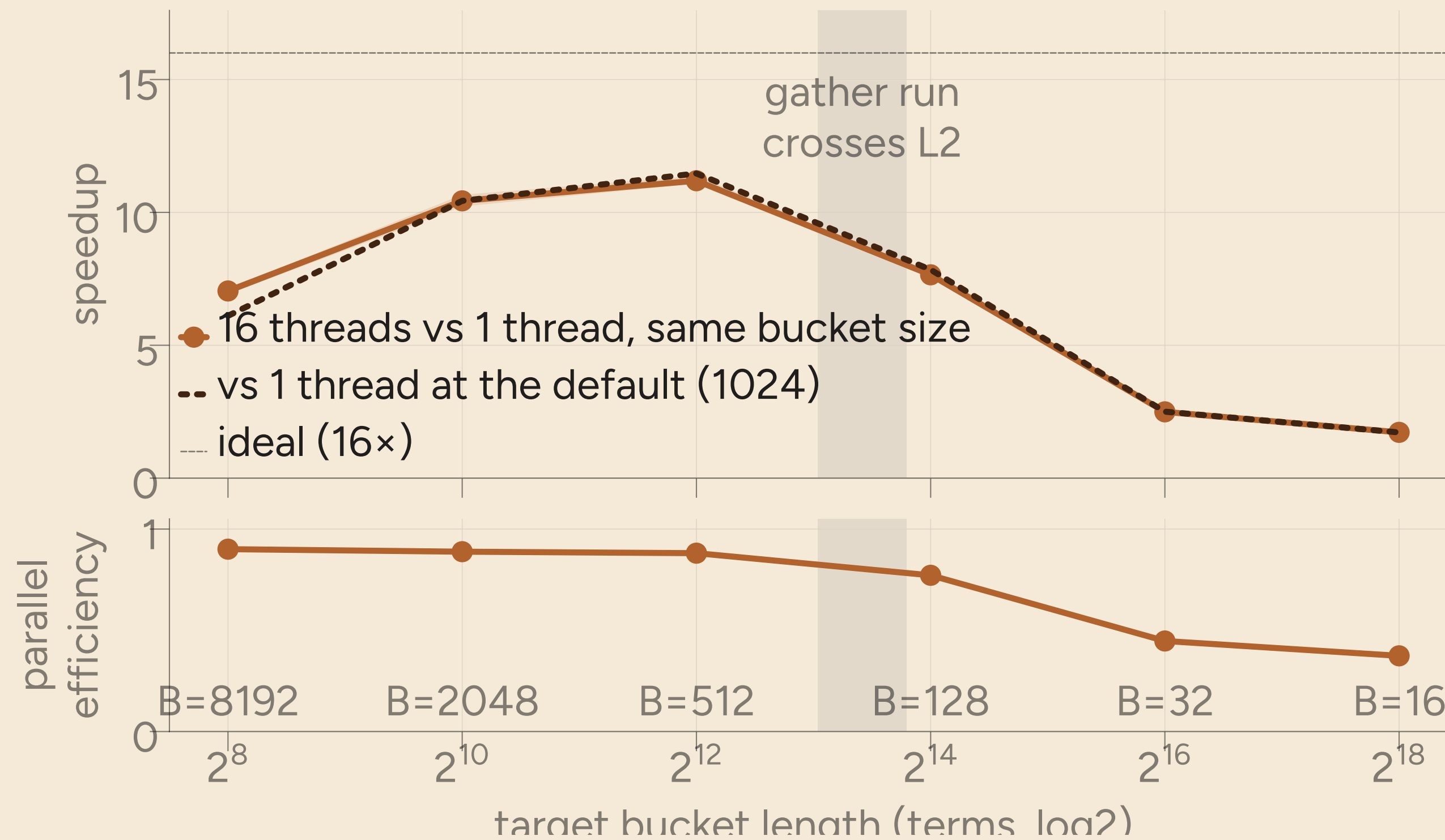


Single-threaded the bucketed engine is already three times faster: there is no global structure to maintain.

10.5x

Sixteen cores on this circuit — eleven to thirteen on pure rotation layers — against the factor of two that both earlier attempts stalled at. Hyperthreads still add nothing.

The bucket size decides whether cores help at all

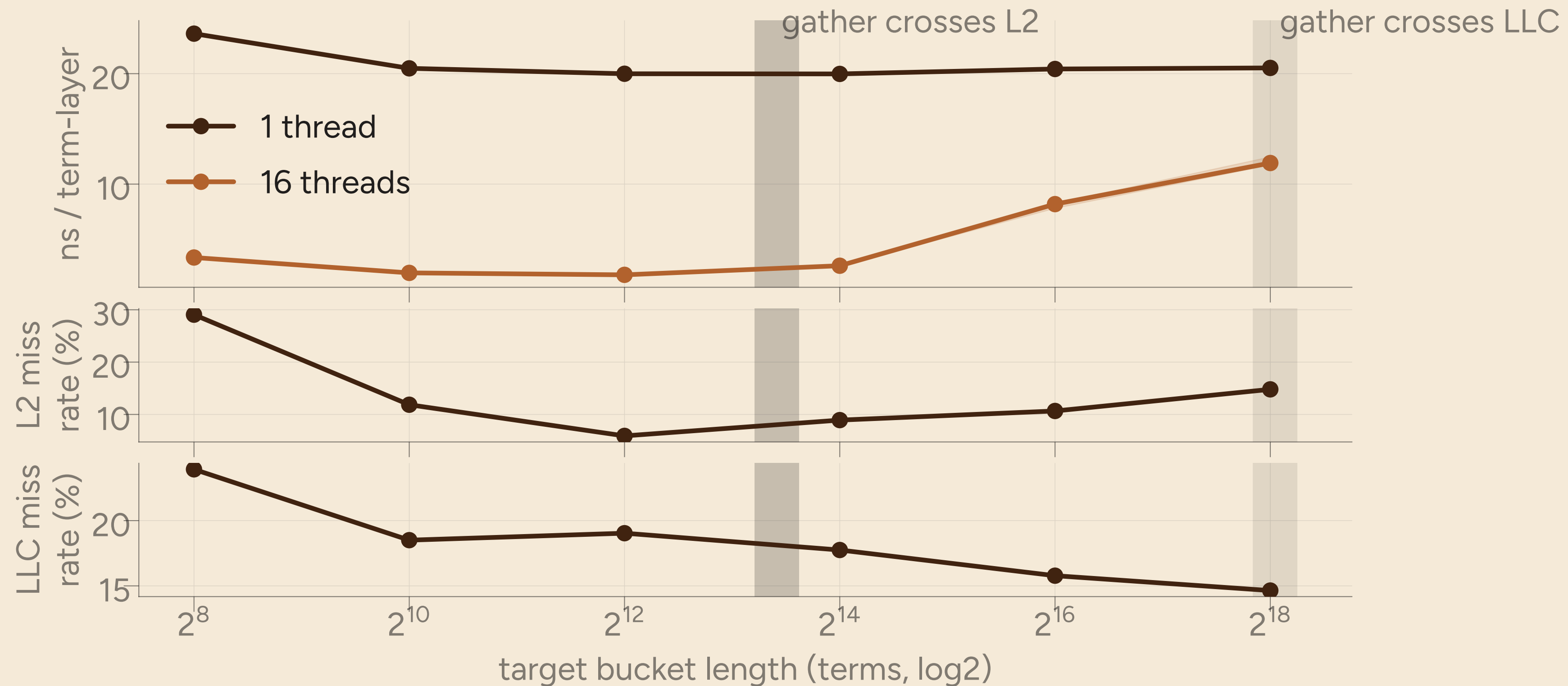


Same code, same circuit, sixteen threads. Below: worker busy time over loop wall $\times$ threads, from the engine's phase counters.

Cores bring cache, not just clocks

- Past about 10^4 terms per bucket the gather run stops fitting L2.
- And there are fewer cosets than threads, so workers starve.
- Sixteen cores are sixteen megabytes of private L2 — buckets are what let the working set live there.

One thread does not care; sixteen care a lot

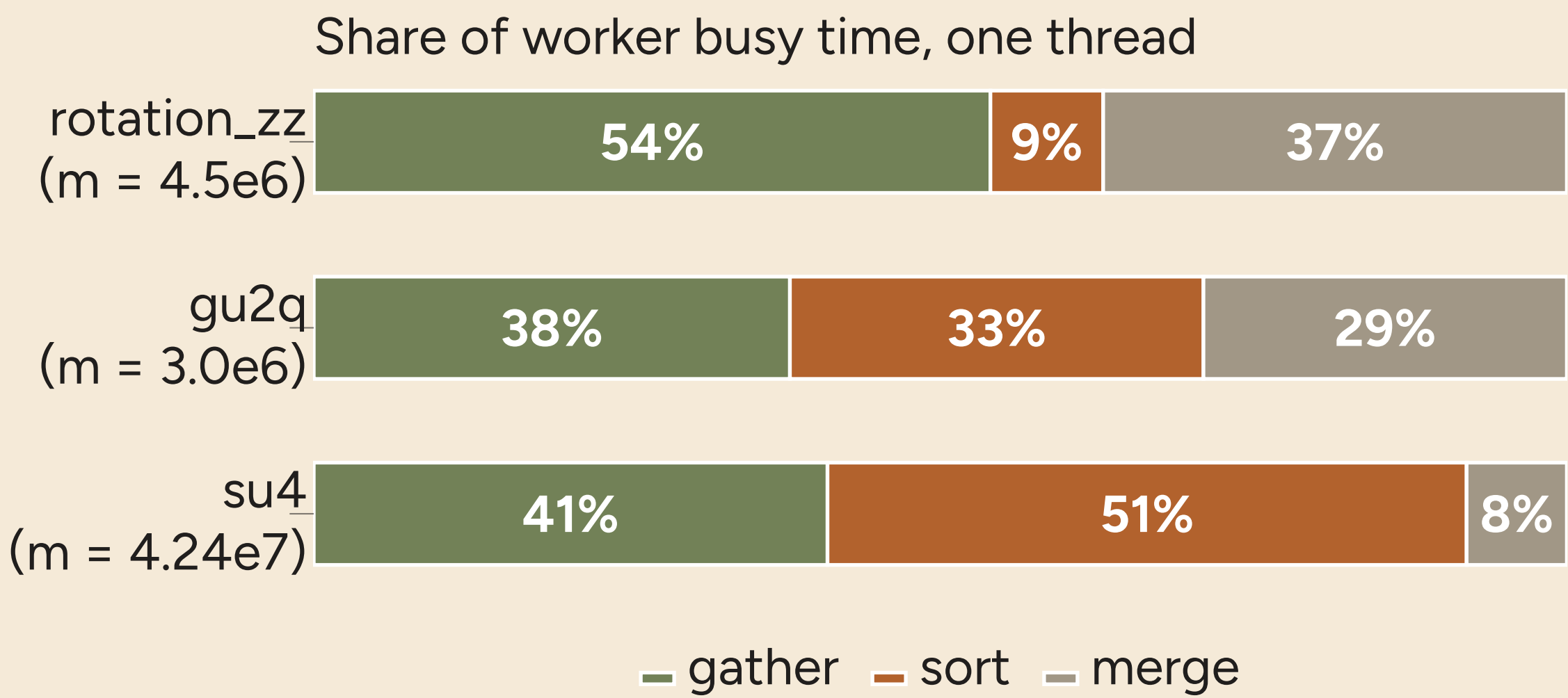


Per-term cost at one and sixteen threads, with single-thread L2 and LLC demand-miss rates from perf stat, against bucket size.

What must fit is the gather run

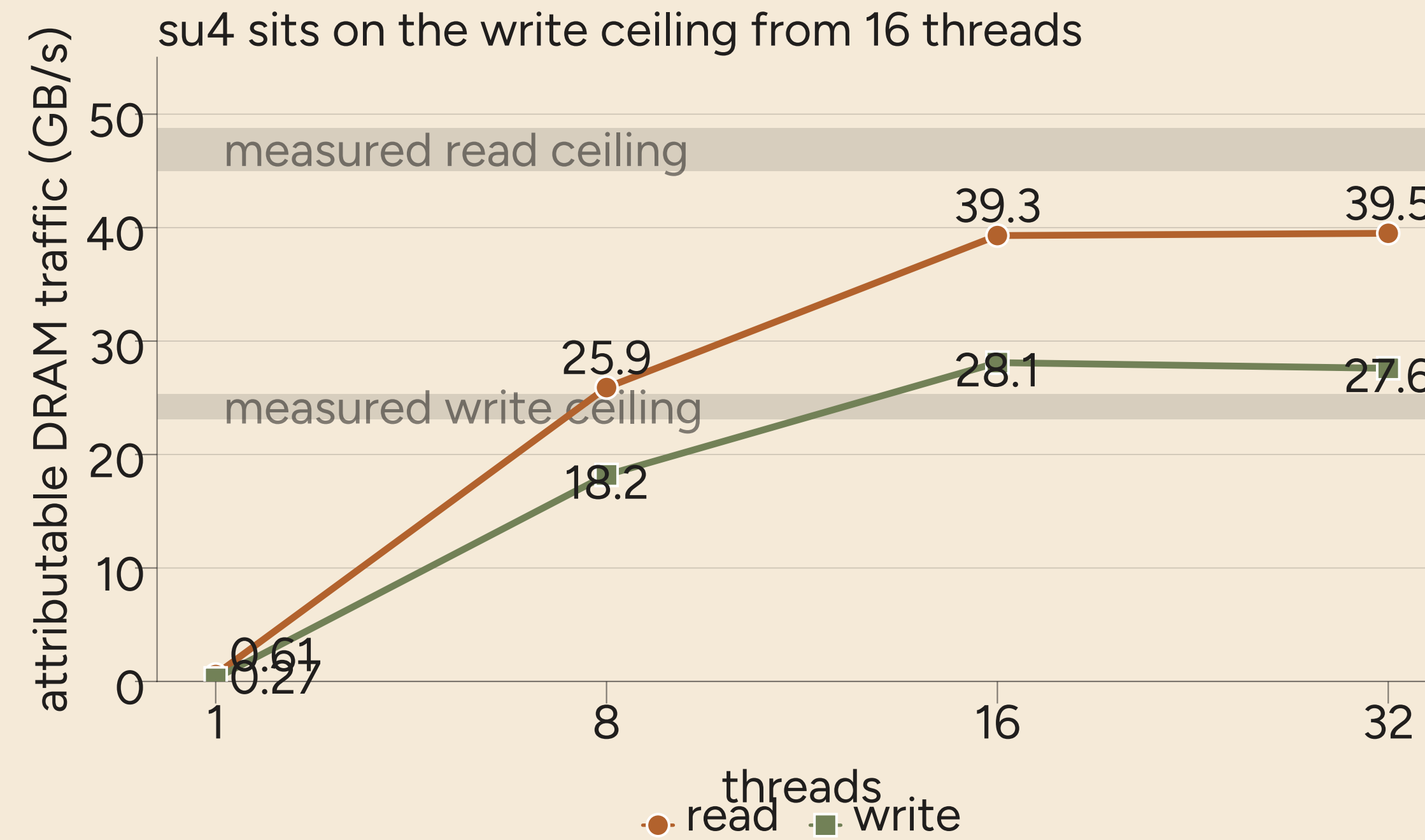
- Fanout $\times$ bucket: $1024 \times 48 \text{ B} \times 2 \approx 100 \text{ KB}$ against a mebibyte of L2.
- Dense two-qubit unitaries have fanout sixteen: 750 KB runs, and the write ceiling at sixteen threads.
- Same algorithm, same bytes — only the order in which they are touched changed.

Where the time goes now



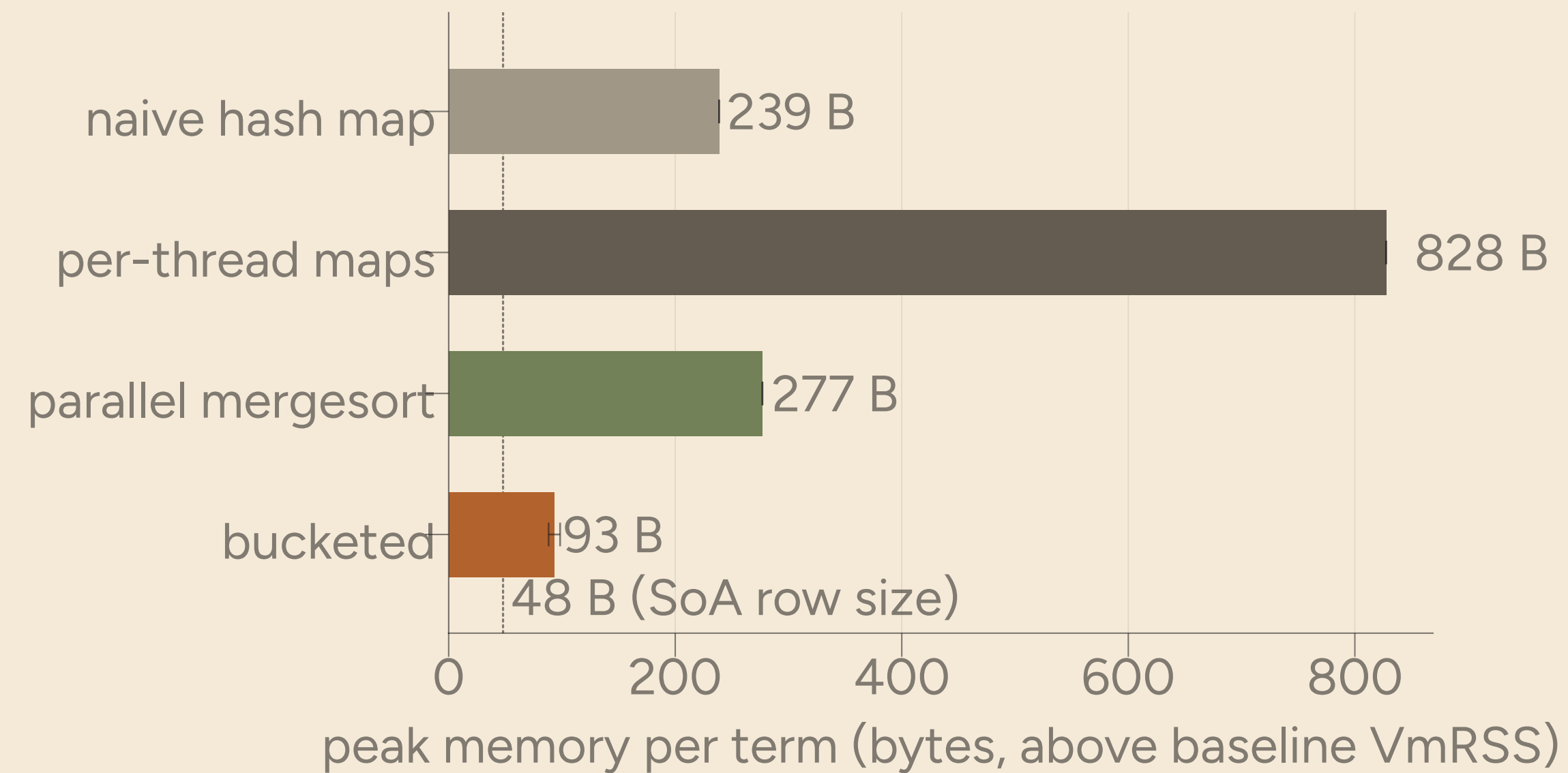
Share of worker busy time per phase, three layer classes, single-threaded. Gather dominates the sparse layers; the dense PTM is sort-bound.

Against the measured ceilings



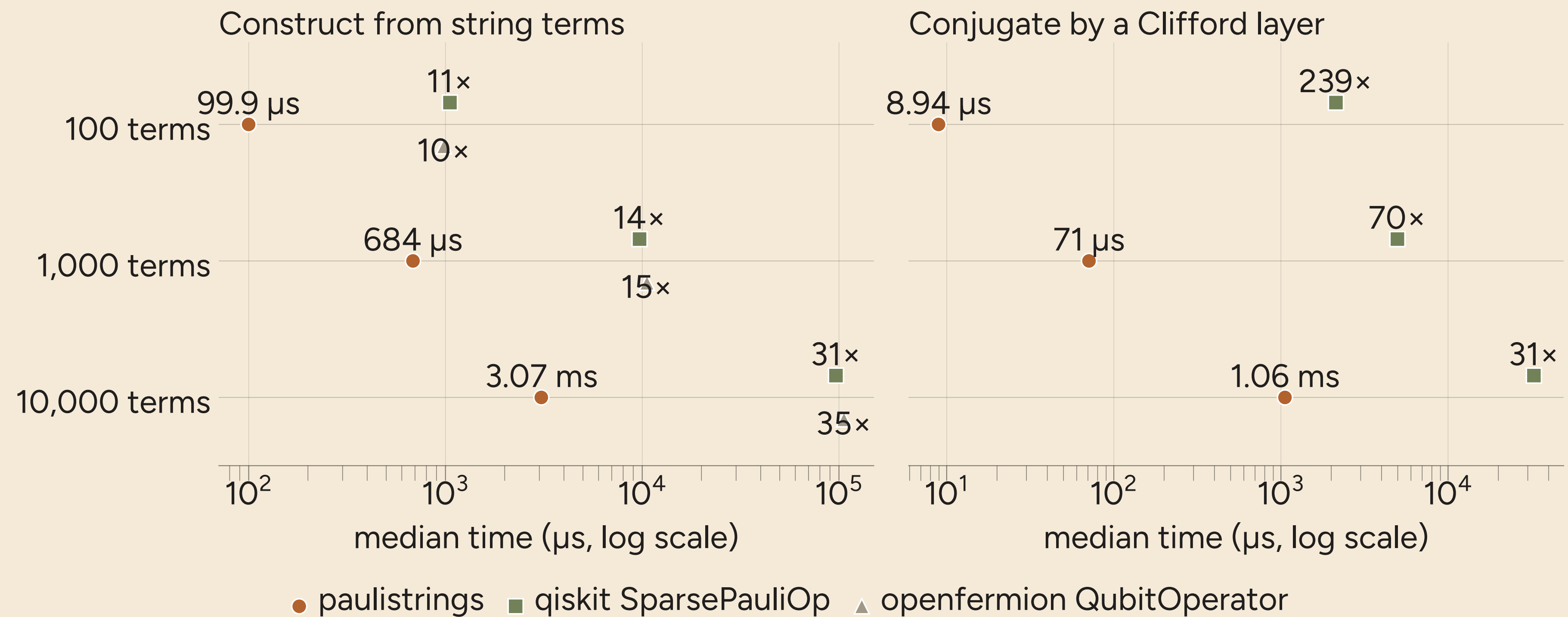
Sparse layers are latency-bound at half the write ceiling even at thirty-two threads; only dense PTMs reach it. The single-thread byte model over-counts DRAM traffic by 2.5–13×: the design's whole point.

Footprint



Peak RSS above the process floor, per peak term. Structure of arrays, no pointers, capacity retained across layers: the steady state allocates nothing.

Against the libraries people use



Construction and Clifford conjugation. Single-threaded against PauliPropagation.jl: two to three times faster above a few thousand terms, five times less memory per term.

04

The gift that keeps on giving

One structure, many wins

- **Locality** — cache-sized runs, and cores that bring cache.
- **Footprint** — bucket-local scratch, no global sort buffers.
- **GPU-ready** — `#[repr(C)]` columns and a fixed fanout.
- **A communication plan** — the same partition, one level up.

The partition is a communication plan

Partition on h across ranks and a layer becomes a sparse, statically known exchange, computed before any term is touched. No key is ever split across ranks.

4

of 271 layers per step cross a boundary — 139 with random rows

21 %

faster per step at two partitions than one process

1

partition per MPI rank, running the same layer loop

Still open

- A bits all-reduce on every layer, and ingestion that replicates the input per rank.
- The dense-PTM write ceiling: smaller gather runs, or fused passes.
- Rank-deficient hash seeds, and a channel-aware bucket floor.

Wie sterk is, moet ook slim zijn.

— strong is 45 GB/s and every byte of L2; smart is knowing where everything goes before you move it

Dank u wel

github.com/lkdvos/paulistrings-rs — Rust core, Python bindings,
and every figure in this talk reproducible from the repository.

Lukas Devos · ldevos@flatironinstitute.org

05

Appendix

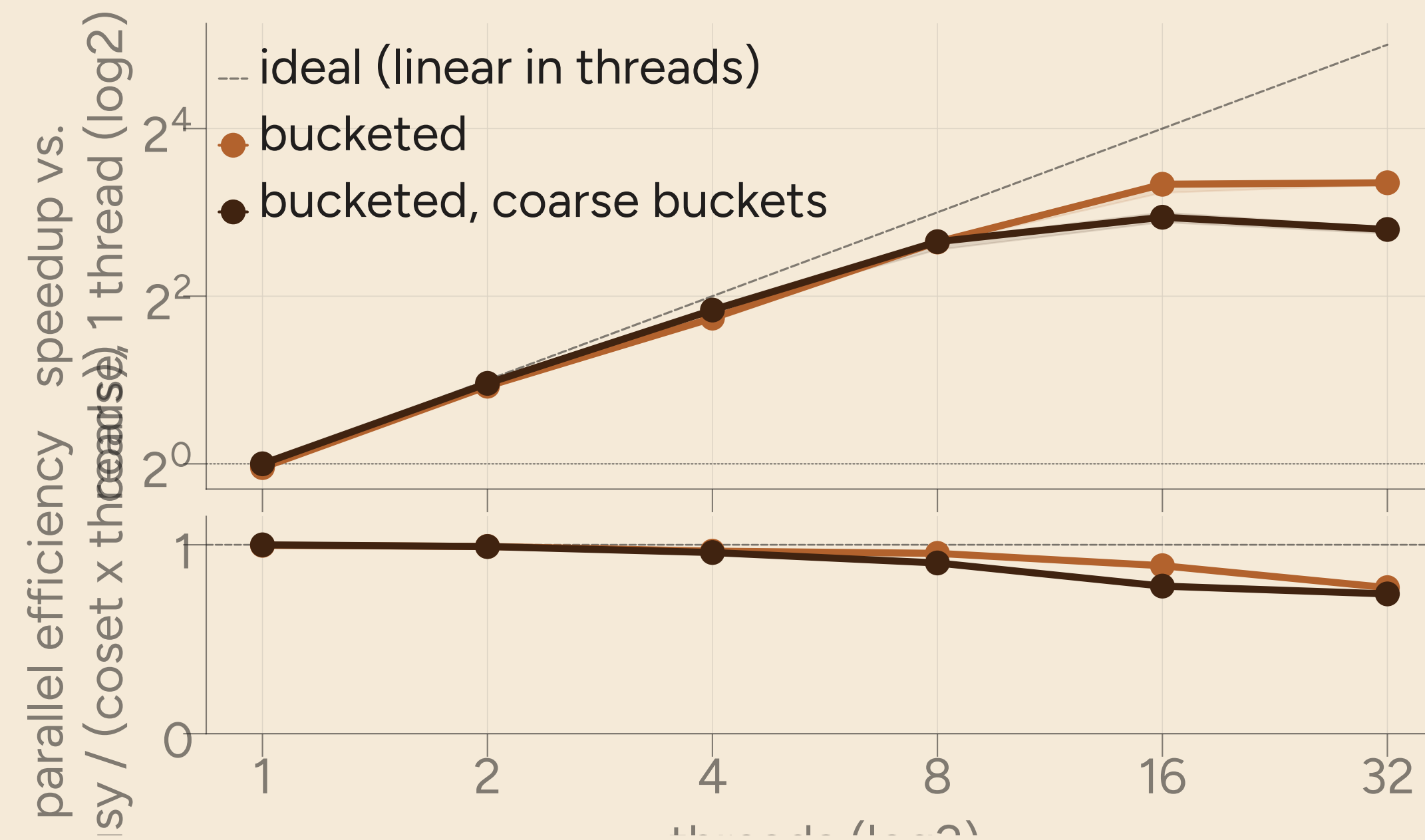
Things that did not work

- **Bucket by support bits and concatenate** — provably wrong: buckets interleave. A four-term counterexample led to the linear hash.
- **Static coset → worker placement** — 1.25–1.9× slower than work stealing in seven of eight cells.
- **Three gather/merge variants** — recompute-in-merge, segment-copy, interleaved keys: all rejected by paired A/B.

Things that did not work

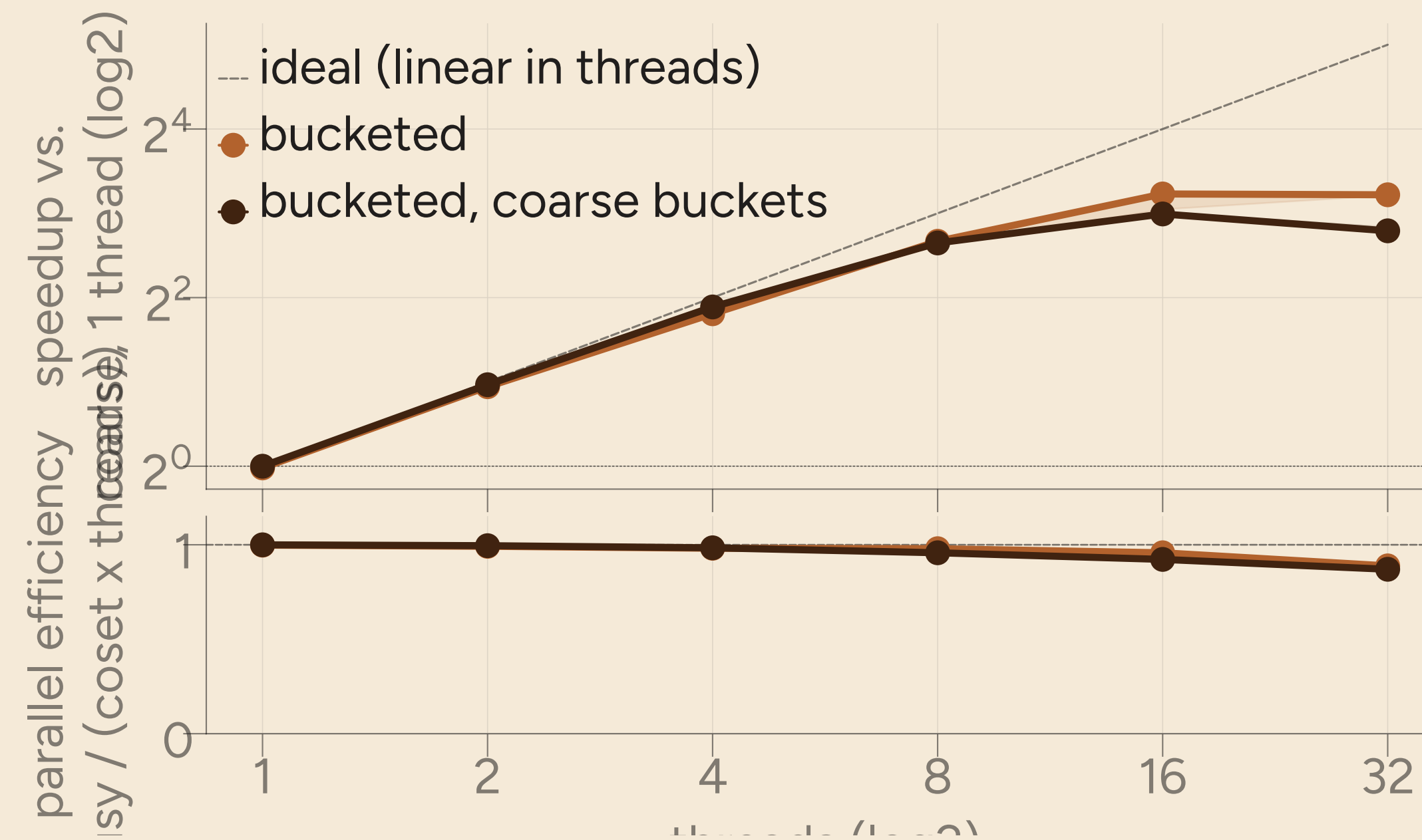
- **Reserving a safe upper bound for merge output** — $2.5\times$ worse peak RSS. The merge's job is to shrink.
- **LTO code layout is real** — one `#[inline]` hint moves the merge kernel by 6–34 %, in both directions.
- Every one of these is written up in `research/notes/`, with the numbers.

Coarse against default buckets



$1.07 \cdot 10^6$ peak terms; coarse is 16 384 terms per bucket, sixty-four buckets. Normalised to the coarse single-thread run.

And at four times the terms



$3.9 \cdot 10^6$ peak terms at $\varepsilon = 2^{-13}$. With enough cosets the coarse arm keeps up: no superlinear regime on this host and circuit.

Reproducing every number

- **Host** — ccqlin038, two Xeon Gold 6244, sixteen cores, two NUMA nodes, 1 MiB L2 per core.
- **Circuit** — `heavy_hex_kicked_ising(127, 10, 5 $\pi$ /16, - $\pi$ /2)`, observable Z_{62} , Heisenberg, `CoefficientThreshold(2-12)`.
- **Baselines** — presentation/bench, each gated against propagate to 10^{-9} .
- **Protocol** — `RUST_LOG` unset, one warm-up, medians of repeated runs, paired A/B for anything under 10 %.